

</> fn main()

Proyecto personal · Sistemas de bajo nivel

Construir un JDK desde cero en Rust

Roadmap técnico por hitos

Autor: **Esteban Nicolás Larena**

Fecha: **22 de julio de 2026**

Contenido

Introducción y alcance	3
Los tres pilares	3
El problema del bootstrap	3
Orden de construcción	4
Fase A — La JVM (el runtime)	5
Concurrencia — ruta al multithread profesional	7
Fase B — El compilador (javac, en Rust)	9
Fase C — Las bibliotecas	10
Fase D — Herramientas (la “DK”)	11
Fase E — Cerrar el círculo	11
Más allá del JDK — la plataforma	13
Fase F — burst (optimizador)	13
Fase G — plain_data / value types	14
Estado actual	15

Introducción y alcance

Este documento traza la ruta para construir un **JDK educativo desde cero en Rust**: no para competir con HotSpot, sino para **cargar y ejecutar bytecode real** y, eventualmente, compilar y correr código Java escrito y compilado con herramientas propias. El objetivo es doble: aprender sistemas de bajo nivel a fondo y completar un reto de ingeniería de gran calibre.

Un JDK no es una sola pieza. Son **tres pilares** que se sostienen entre sí:

Los tres pilares

Pilar	Qué hace	Lenguaje
JVM (el <i>runtime</i>)	Carga y ejecuta bytecode. Parser de <code>.class</code> → intérprete → objetos → heap → GC.	Rust
Compilador (<code>javac</code>)	Convierte <code>.java</code> en bytecode <code>.class</code> : lexer → AST → análisis semántico → emisión.	Rust
Bibliotecas (<code>java.base</code>)	<code>Object</code> , <code>String</code> , <code>System</code> , colecciones... escritas <i>en Java</i> , compiladas por el compilador propio.	Java

El problema del bootstrap

Las bibliotecas necesitan al **compilador** (para compilarse) y a la **JVM** (para ejecutarse). Si el compilador se escribiera en Java, se necesitaría a sí mismo para compilarse — el clásico problema del huevo y la gallina.

*Decisión de diseño: el compilador se escribe **en Rust**, igual que la JVM. Así Rust compila al compilador, el compilador compila las bibliotecas, y la JVM las ejecuta. El ciclo queda roto.*

Orden de construcción

La **JVM va primero**, sin excepción: tanto la salida del compilador como las bibliotecas son inertes sin un motor que las ejecute, y no se puede ni *probar* que el compilador genera bytecode correcto sin algo que lo corra.

A · JVM → B · Compilador → C · Bibliotecas → E · Cerrar el círculo
(motor) (.java → .class) (en Java) (todo junto)
└─ D · Herramientas se completan en el camino (javap, java, jar...)

Horizontes: **Base** el núcleo, por aquí empezamos **Avanzado** más duro, segunda pasada **Cumbre** lo más difícil, pero vamos a llegar

*Cómo leer el roadmap: es una **ruta ordenada por hitos**. Cada hito tiene un criterio de éxito medible y no se cierra hasta cumplirlo. El orden respeta las dependencias: no se puede el hito N sin el N-1.*

Fase A — La JVM (el motor que ejecuta bytecode)

A0 Parsear el .class (≡ javap) ✓ núcleo logrado Base

Base Lector de bytes (cursor big-endian) — el `Reader` (`u1` / `u2` / `u4`)

Base Constant pool (17 clases de entrada; 1-indexed, Long/Double = 2 slots)

Base Header: magic, versiones, flags, this/super/interfaces

Base fields, methods, attributes: `Code`, `LineNumberTable`, `SourceFile`, `StackMapTable` (los 7 frame types)

Base Desensamblado de bytecode (tabla de opcodes completa) con comentarios `// ...` resueltos

Base Volcado `javap` brief y `-v` byte-identico; flags de visibilidad (`-public` / `-protected` / `-package` / `-p`)

Avanzado *Pendiente (no esencial)*: `Signature`, `BootstrapMethods`, `InnerClasses`, `Exceptions`, `ConstantValue`, anotaciones, `Record`, `NestHost/Members`, `LocalVariableTable`

✓ **Éxito: logrado** — el volcado coincide **byte a byte** con `javap -v` (y brief) sobre 12 fixtures.

A1 Intérprete mínimo ✓ logrado Base

Base *Frame*: pila de operandos + variables locales

Base Contador de programa (PC) y *loop* de despacho de opcodes

Base Opcodes: `iconst`, `iload`, `istore`, `iadd`, `ireturn`

Base Parseo de descriptores de método (`(II)I`)

✓ **Éxito: logrado** — ejecutar un método que suma dos enteros (`Add.add`).

A2 Control de flujo y métodos ✓ logrado Base

Base Saltos: `if_icmpgt`, `goto`, comparaciones

Base *Method area* (metadatos de clases cargadas)

Base `invokestatic` + pila de frames (llamadas anidadas)

✓ **Éxito: logrado** — corren `factorial` recursivo (120) y `fibonacci` (55).

A3 Objetos y heap ✓ logrado Base

Base Heap + allocator simple; arena de bytes (`Vec<u8>`) + `malloc` bump

Base *Layout* de objetos (campos, con herencia) y de clases (con *vtable*)

Base `new`, `getfield` / `putfield`

Base `invokevirtual` / `invokespecial` / `invokeinterface` + dispatch dinámico (*vtable* + *itable*)

Base Arrays (objetos y primitivos int-category, con ancho fiel)

✓ **Éxito: logrado** — herencia (`Dog extends Animal`) y dispatch dinámico verificados: `a.sound()` sobre una referencia `Animal` / `Speaker` corre `Dog.sound`.

A4 Robustez del runtime ✓ **logrado** **Base**

Base Excepciones: `throw` + tablas de excepción + *stack unwinding*; `finally`; excepciones **implícitas** (NPE, índice, cast, tamaño negativo)

Base *Linking*: resolución bajo demanda + **verificador** (type-checking con *StackMapTable*); errores de linkage (`NoClassDefFound` / `NoSuchMethod`)

Base Inicialización de clase (`<clinit>` , perezosa, super-primero)

Base Jerarquía de class loaders (bootstrap/app + delegación)

✓ **Éxito: logrado** — un `try / catch` atrapa una `Boom` (con unwinding entre frames y `finally`), y `Base / Derived` inicializan en orden super-primero.

A5 Nativos + GC ✓ **logrado** **Avanzado**

Avanzado Puente a métodos nativos (I/O real) — `System.out.println` imprime de verdad

Avanzado *Intrinsics* — `getClass` , `hashCode` , `Math` , `arraycopy` , `String / Class` ...

Avanzado GC mark & sweep — y más: **compactante**, free list con *coalescing*, política de fragmentación, 4 disparadores sobre *safepoint*, y **marcado transitivo** correcto

✓ **Éxito: logrado** — `System.out.println` imprime de verdad y el GC recolecta (y **compacta**) basura.

A6 Cumbre del runtime 🚧 **en progreso** **Cumbre**

Cumbre ✓ **Verificador de bytecode JVMMS-estricto** — verifica **con o sin** `StackMapTable` (inferencia por punto fijo como fallback, y *cross-check* de la tabla contra la inferencia); gate **estructural** (§4.9: targets, no caerse del final, `jsr` / `ret`, tabla de excepciones); tipos de locales estrictos + cota de `max_stack`; reglas de **objetos sin inicializar** (`<init>` una sola vez); **handlers de excepción** tipados; y **acceso/linkage** (regla `protected`, `count` de `invokeinterface`)

Cumbre ✓ **Sistema de tipos completo** — `int` / `long` / `double` / `float` ejecutados y verificados: cómputo, conversiones, comparaciones, división con excepción, categoría-2 (params/campos/estáticos/arrays/frames), y el **lattice de referencias** (covarianza de arrays + `join` / LUB)

Cumbre ✓ **GC generacional** — young (Eden + 2 *survivors*) con colector por **copia** (minor: evacúa/promueve por edad, estilo Cheney con *forwarding*), **write barrier + remembered set** para las raíces `old→young`, y Old con **mark-sweep-compact**; *minor* disparado al llenarse Eden, *major*/full por occupancy o explícito

Cumbre ✓ **Referencias débiles** (`java.lang.ref`) — `WeakReference` + `ReferenceQueue`: el major trata `referent` como débil y, al morir el referente, lo limpia (`get()→null`) y **encola** la referencia. Base para `PhantomReference` / `Cleaner` (post-mortem)

Cumbre **Hilos / concurrencia** (*en progreso*) — objetivo: **multithread profesional**, por fases. ✓ **Fase 0 — green threads**: `java.lang.Thread` + `start` / `join` / `sleep`, scheduler cooperativo round-robin en el safepoint, varios call stacks por-thread, GC sobre las raíces de **todos** los threads, terminación; verificado (`Threads.run → 100`) y visible en el step-visualizer. ✓ **Monitores**: `synchronized` de **bloques y métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma/suelta el monitor al entrar/salir del frame), reentrancia, señalización `wait` / `notify` / `notifyAll` sobre *wait-set/blocked-set*, `IllegalMonitorStateException`, `wait(timeout)` y monitor **GC-safe** (las claves se remapean por el *forward* del GC, así que se compacta con monitores tomados). ✓ **Substrato de SO + GIL (E1+E2)**: `JVM_THREADS=os` corre cada `Thread.start()` en un `std::thread` real tras un **GIL** (`Arc<Mutex<JVM>>`, un opcode por lock), con `park` / `unpark` reales y `sleep` en tiempo real — correcto pero todavía **serializado**. **Falta**: la **API de Thread** completa (`interrupt`, `currentThread`, `yield`, `getState`, `daemon` — hito H1, el próximo) y **sacar el GIL** para paralelismo real (E3). El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA), no en el heap Java. *La hoja de ruta completa (JMM, CAS, `java.util.concurrent`) tiene su propia sección («Concurrencia»)*.

Cumbre ✓ **Cobertura del set de opcodes — 199/199 alcanzables: completo** — cerrados `nop` (0x00), `goto_w` (0xc8), el prefijo `wide` (0xc4: índice de local de 16 bits, la única forma de direccionar los slots pasados el 255; `wide iinc` mide 6 bytes, el resto 4 — salió barato porque los handlers de locales ya reciben `slot: usize` y nunca se enteran del ancho, así que ensanchar es puro *decoding*) y `multianewarray` (0xc5: aloca recursivamente **sólo** los `dimensions` niveles indicados — `new int[3][ ]` deja los internos en `null` —, validando todos los conteos *antes* de alocar nada; los niveles superiores guardan referencias y sólo el más interno usa el ancho real del elemento, y las referencias hijo pasan por `store_reference` para que el *remembered set* vea los punteros `old→young`). Hubo que **modelarlo también en el verificador**, sin lo cual la clase corría con el verificador claudicando en silencio). Los 3 de `jsr` / `ret` / `jsr_w` quedan **excluidos por diseño** — **JVMS §4.9.1** los prohíbe en class files de versión 50.0+ (Java 6 en adelante)— con la postura **leer sí, ejecutar no**: el desensamblador los soporta completo (requisito de A0, que se mide contra `javap` sobre `java.base`) y el gate estructural del

verificador los rechaza. Nota de diseño: `subrutinas-jsr-ret.md`. El número honesto es entonces **199 de 199 alcanzables: completo**

Cumbre ✓ `invokedynamic` (0xba) — **no era un opcode, era un subsistema**, y corre: **5 de las 6 fábricas** que emite `javac`. `StringConcatFactory` (todo `"a" + b` desde Java 9), `SwitchBootstraps.typeSwitch` (`switch` sobre patrones de tipo y de enum), `ObjectMethods` (`equals` / `hashCode` / `toString` de un `record`, los tres desde una sola entrada de `BootstrapMethods`, distinguidos por el nombre del call site), `LambdaMetafactory.metafactory` (lambdas y method references) y `ConstantBootstraps.invoke` (constantes dinámicas). La sexta, `altMetafactory`, necesita serialización y queda **fuera de alcance**. Dos decisiones de diseño valen la pena: las lambdas se resuelven con un **objeto sintético** en vez de generar una clase en runtime —lo único donde nuestro diseño es *más simple* que el de HotSpot, porque somos dueños del despacho— y los *bootstrap methods* se modelan como **intrínsecos**, con `java.lang.constant` escrito en Java. Ruta completa, correcciones y mediciones: `invokedynamic-ruta.md`

Avanzado ✓ `ldc` **de literales de clase** (`Foo.class`, `int[].class`) — empuja el mirror `Class<...>`, cacheado por Class ID (así `Foo.class == Foo.class`) y **sin inicializar** la clase: un literal no es *uso activo* (§5.5). Los de array reusan el mirror sintético que arma `anewarray`, ya que una clase de array no tiene `.class` que cargar

Cumbre ✓ **La VM puede invocar Java** (`call_java`) — empuja un frame propio y lo corre con un bucle de `run_one` anidado, devolviendo el resultado. Resultó ser el **caso general de lo que la inicialización de clases ya hacía**: `<clinit>` era la variante sin argumentos ni resultado. Importa porque **los intrínsecos dejan de ser terminales**: un nativo que sólo puede calcular y devolver tiene que reimplementar lo que necesite de la biblioteca. Con esto, `String.valueOf(Object)` llama al `toString()` del objeto, un `record` pregunta el `equals` / `hashCode` de sus componentes, y un `condy` ejecuta su bootstrap

Cumbre  **Modelo de objetos de** `java.lang.invoke` (`MethodHandle` / `MethodType` / `Lookup`) — lo último que le falta a 0xba. Desbloquea `ldc` de esas constantes y los *bootstrap methods definidos por el usuario*. Reparto: las clases en **Java**, pero `MethodHandle.invoke` **obligatoriamente nativo** (invocar un método arbitrario desde un handle es justo lo que Java no puede hacerse a sí mismo — y el JDK real también lo trata como intrínseco de VM). El premio: con eso, `ConstantBootstraps.invoke` deja de ser un intrínseco nuestro y se reescribe en dos líneas de Java. **Medido**: la mitad difícil (la polimorfia de firma de `invoke`) es alcanzable desde Java corriente y por lo tanto testeable **ya**; la otra mitad no tiene oráculo hasta que exista el escritor de `.class` (hito **B3**), porque `javac` nunca emite un `ldc` de esas constantes ni permite invocar un bootstrap propio

Cumbre JIT (bytecode → código nativo)

✓ **Éxito: parcial** — **verificador JVMs-estricto**, **GC generacional**, el **set de opcodes completo** (`invokedynamic` incluido) e **hilos de SO reales tras un GIL** logrados; falta el **paralelismo real** (sacar el GIL), el JMM y/o el JIT.

Concurrencia — ruta al multithread profesional

La ejecución concurrente (Hito A6) merece su propio mapa, porque es donde el proyecto apunta a **multithread profesional**. Lo que hoy corre sobre **green threads** es la *planta baja* de un edificio cuya cumbre es `java.util.concurrent`. Cada planta se apoya en la de abajo: no hay `ReentrantLock` sin CAS, ni CAS útil sin un modelo de memoria, ni modelo de memoria que *importe* sin hilos reales que lo hagan necesario.

RASCACIELOS DE LA CONCURRENCIA · construir hacia arriba ↑

[6]	<code>java.util.concurrent</code>	· lo que usa la gente	FALTA
[]	AQS · <code>ReentrantLock</code> · <code>Condition</code> · <code>Semaphore</code> · <code>Latch</code>		
[]	<code>BlockingQueue</code> · <code>ConcurrentHashMap</code> · <code>Executor/pools</code>		
[5]	Atómicos / CAS	· raíz lock-free	FALTA
[]	<code>compareAndSwap</code> · <code>AtomicInteger</code> / <code>AtomicLong</code> / <code>Reference</code>		
[4]	Modelo de Memoria (JMM)	· visibilidad + orden	FALTA
[]	<code>volatile</code> · <code>happens-before</code> · fences · <code>final</code> · <code>no-tearing</code>		
[3]	API de Thread	· ← ACÁ ESTAMOS (H1)	PARCIAL
[x]	<code>start</code> · <code>run</code> · <code>join</code> · <code>sleep</code> (wall-clock en modo OS)		
[]	<code>interrupt</code> · <code>yield</code> · <code>currentThread</code> · <code>getState</code> · <code>daemon</code>		
[2]	Monitor intrínseco	· monitor completo	CASI
[x]	<code>monitorenter/exit</code> · <code>synchronized</code> (bloques Y métodos)		
[x]	reentrancia · <code>wait</code> / <code>notify</code> / <code>notifyAll</code> · IMSE		
[x]	<code>wait(timeout)</code> · GC-safe (sobrevive minor/compact)		
[]	<code>interrupt-of-wait</code> (→ H1) · <code>biased/thin locks</code>		
[1]	Substrato de ejecución	· paralelismo real	PARCIAL
[x]	green threads (cooperativo, default + visor)		
[x]	hilos de SO reales + GIL · <code>park/unpark</code> · GC bajo GIL		
[]	sacar el GIL (E3) · TLABs · STW handshake · preempción		

▲ todo descansa en la planta 1 ▲

Estado:  hecho  parcial  falta

Planta	Estado	Qué falta para “profesional”
6 · <code>java.util.concurrent</code>	 falta	AQS (<code>AbstractQueuedSynchronizer</code> , base de casi todo), <code>ReentrantLock</code> / <code>ReadWriteLock</code> , <code>Condition</code> , <code>Semaphore</code> , <code>CountDownLatch</code> , <code>CyclicBarrier</code> , <code>BlockingQueue</code> , <code>ConcurrentHashMap</code> , <code>Executor</code> /pools, <code>CompletableFuture</code>
5 · Atómicos / CAS	 falta	<code>compareAndSwap</code> (la instrucción atómica raíz) y los <code>Atomic*</code> . Sin esto no hay lock-free ni AQS .
4 · Modelo de Memoria (JMM)	 falta	<code>volatile</code> , <i>happens-before</i> , barreras/fences, semántica de <code>final</code> , no-tearing de <code>long</code> / <code>double</code> .
3 · API de <code>Thread</code>	 parcial — ← acá (H1)	Hecho: <code>start</code> / <code>run</code> / <code>join</code> / <code>sleep</code> (ya <i>wall-clock</i> en modo OS). Falta: <code>interrupt</code> / <code>InterruptedException</code> , <code>yield</code> , <code>currentThread</code> , <code>Thread(Runnable)</code> , nombre/id, <code>isAlive</code> , <code>getState</code> , prioridades, daemon. Hoy <code>ThreadStatus</code> tiene cuatro estados (<code>Runnable</code> / <code>Blocked</code> / <code>Waiting</code> / <code>Terminated</code>): faltan <code>NEW</code> y <code>TIMED_WAITING</code> — este último modelado con el campo aparte <code>sleep_until</code> .
2 · Monitor intrínseco	 casi	Hecho: <code>monitorenter</code> / <code>exit</code> , <code>synchronized</code> (bloques y métodos), reentrancia, <code>wait</code> / <code>notify</code> / <code>notifyAll</code> , <code>IllegalMonitorStateException</code> , <code>wait(timeout)</code> y monitor GC-safe (las claves se remapean por el <i>forward</i> del GC en minor/compact, así que se puede compactar con monitores tomados). Falta: interrupción del <code>wait</code> (llega con H1), biased/thin locks, detección de deadlock.
1 · Substrato de ejecución	  gran avance	Hecho (E1+E2): green threads (scheduler cooperativo, default y visor) y hilos de SO reales (<code>std::thread</code> por <code>Thread.start()</code>) serializados por un GIL (<code>Arc<Mutex<JVM>></code> , 1 opcode por lock); <code>park</code> / <code>unpark</code> reales; GC seguro bajo el GIL (stop-the-world implícito). Falta (E3): sacar el GIL → paralelismo real (locks finos + TLABs + handshake STW), preempción.

Los dos cimientos reales

De todo el edificio, **dos primitivas** sostienen el resto: `park` / `unpark` (planta 1) y **CAS** (planta 5). De esas dos se deduce *casi todo* lo de arriba — AQS, los locks, los atómicos, los pools. Por eso el salto grande **no** era terminar el monitor (planta 2), sino cambiar el **substrato** de green threads a hilos de SO — y ese salto **ya está dado** (E1+E2): `park` / `unpark` son reales y el monitor quedó cerrado salvo la interrupción del `wait` . Queda la segunda mitad: **sacar el GIL** (E3), y después CAS como cimiento de todo lo lock-free.

Ruta crítica



El camino canónico — el mismo que recorrieron CPython y las JVM tempranas — es **GIL primero** (hilos de SO con un lock global que serializa el intérprete: simple y correcto, pero todavía sin paralelismo real) y después **sacar el GIL** (locks finos por estructura + TLABs + GC stop-the-world). Recién ahí `volatile` y los *fences* dejan de ser decorativos: con green threads no hay reordenamientos reales que tapar; con hilos de SO, omitirlos rompe el código de formas no-deterministas.

*Para el LLM, el paralelismo pesado **no** vive en este rascacielos: vive en **kernels off-heap** (rayon/CUDA). Esta torre es para la **corrección** de la concurrencia Java, no para el cómputo numérico — que sale del heap por completo.*

Fase B — El compilador (javac, escrito en Rust)

El front-end convierte texto en estructuras y la back-end las convierte en bytecode:

.java → lexer (tokens) → parser (AST) → análisis semántico → generación de bytecode → .class

B0 Lexer Base

Base Scanner: texto `.java` → tokens (keywords, identificadores, literales, símbolos)

✓ **Éxito:** tokeniza `Add.java` sin perder ni inventar tokens.

B1 Parser Base

Base Gramática → AST (clases, métodos, sentencias, expresiones)

Base Tabla de símbolos / *scopes*

✓ **Éxito:** produce un AST correcto de `Add.java`.

B2 Análisis semántico Base

Base Resolución de nombres

Base *Type checking*

✓ **Éxito:** acepta `Add.java` y rechaza un programa con error de tipos.

B3 Generación de bytecode Base

Base AST → bytecode

Base Construcción del constant pool

Base Escritor de `.class`

✓ **Éxito:** tu `javac` compila `Add.java` y el `.class` corre en **tu** JVM con el mismo resultado que el real.

B4 Compilador robusto Avanzado

Avanzado *Overload resolution* y chequeo de *override*

Avanzado Análisis de flujo (asignación definitiva, alcanzabilidad)

Avanzado Inferencia de tipos (`var`)

Avanzado Generación de `StackMapTable`

✓ **Éxito:** compila programas con sobrecarga, herencia y flujo no trivial.

B5 Cumbre del compilador Cumbre

Cumbre Genéricos completos (*type erasure*, *wildcards*, inferencia)

✓ **Éxito:** compila código genérico equivalente al de `javac`.

Fase C — Las bibliotecas (en Java, compiladas por tu javac)

C0 Núcleo de java.lang Base

Base `Object`, `String`, `System`, `wrappers`, `Math`, `StringBuilder`

✓ **Éxito:** un programa que usa `String` y `System.out` corre en tu JVM.

C1 Excepciones Base

Base Jerarquía `Throwable` / `Exception` / `RuntimeException`

✓ **Éxito:** lanzar y atrapar excepciones de la biblioteca propia.

C2 Colecciones e IO Avanzado

Avanzado `java.util`: `List`, `ArrayList`, `Map`, `HashMap`

Avanzado `java.io`: `InputStream` / `OutputStream` / `PrintStream`

✓ **Éxito:** un programa que usa `ArrayList` / `HashMap` corre.

C3 Cumbre de la biblioteca Cumbre

Cumbre Resto de `java.base` (`net`, `nio`, `time`, reflexión completa...)

✓ **Éxito:** cubrir lo necesario de `java.base` para programas reales.

Fase D — Herramientas (la “DK” = Development Kit)

No se construyen en bloque, sino que se van completando como subproducto de las otras fases.

Herramienta	Cuándo se logra	Horizonte
<code>javap</code> (desensamblador)	Se logra con el Hito A0	Base
<code>java</code> (lanzador de la JVM)	Se logra durante la Fase A	Base
<code>javac</code> (compilador)	Es toda la Fase B	Base
<code>jar</code> (empaquetador de <code>.class</code>)	Tras la Fase B	Avanzado
<code>jdb</code> , <code>javadoc</code> , <code>jlink</code> , <code>keytool</code>	Largo plazo	Cumbre

Fase E — Cerrar el círculo

El momento épico: las tres piezas funcionando juntas.

- **Cumbre** Compilar las bibliotecas (Fase C) con tu propio `javac` (Fase B)
- **Cumbre** Ejecutar un programa real que use esas bibliotecas en tu propia JVM (Fase A)
- **Cumbre** Conformance: comportamiento fiel a la especificación

✓ **Éxito:** un `.java` que escribís → lo compila tu `javac` → usa tus bibliotecas → corre en tu JVM, sin tocar nada del JDK de Temurin.

Más allá del JDK — la JVM como plataforma

La JVM no es un fin en sí mismo: es la **carrocería** de un proyecto más grande. Como construimos VM + compilador propios, controlamos el stack entero y podemos **inventar** más allá de lo que `javac` / HotSpot permiten. Estos dos tracks son extensiones aspiracionales apoyadas en las fases A–E.

Fase F — `burst`: el optimizador (rendimiento)

Módulo de optimización **opcional** atornillado sobre el intérprete ingenuo, que actúa de **chasis y oráculo de corrección**. `burst` debe dar resultados **idénticos** y se valida con **differential testing** (mismo programa por los dos caminos → `assert` de igualdad). Importante: un optimizador **no necesita JIT** — el intérprete tiene su propia caja de herramientas (típico 2–10× sin compilar a nativo).

F0 Quickening Avanzado

Avanzado Resolver refs del constant pool **una sola vez** y reescribir el opcode a su variante resuelta (como las JVM reales)

✓ **Éxito:** un método que invoca en bucle no re-resuelve el constant pool en cada vuelta.

F1 Superinstrucciones / fusión Avanzado

Avanzado Fusionar `iload`, `iload`, `iadd` en un solo handler (primo del *operator fusion* de los compiladores ML, p. ej. FlashAttention)

✓ **Éxito:** menos overhead de despacho fusionando secuencias calientes.

F2 Inline caching + stack caching Cumbre

Cumbre Inline cache en sitios de llamada (recordar receptor → método)

Cumbre Tope de la pila de operandos en registro/variable

✓ **Éxito:** dispatch dinámico acelerado en llamadas repetidas.

F3 JIT (bytecode → nativo) Cumbre

Cumbre Compilación de métodos calientes con *register allocation*, etc.

✓ **Éxito:** compilar métodos calientes a nativo. La cumbre lejana — **meta real** del track (apuesta del dueño), no descartada; el differential testing la mantiene honesta.

Fase G — `plain_data` / value types (modelo de datos)

Los **“huérfanos de Object”**: tipos planos sin header, sin identidad, sin monitor, flatteables. Viable porque tenemos compilador propio (Fase B) que puede emitir el constructo y una VM que lo trata especial. Es, en chiquito, el principio de representación de un **tensor** — puente directo a sistemas de ML. Inspiración: value classes de Valhalla, `struct` de .NET.

G0 `plain_data` plano en el heap Cumbre

Cumbre Tipo sin header: `[field0 | field1]` vs objeto normal `[class_ptr | fields]`

Cumbre El compilador (Fase B) lo marca; la VM lo guarda inline

✓ **Éxito:** un value type sin header conviviendo con los objetos normales.

G1 Arrays flatteados + semántica de valor Cumbre

Cumbre `plain_data[]` contiguo: sin los N punteros, ni los N headers, ni N alocaiones

Cumbre Igualdad por valor (comparar bytes), sin `null`, sin GC para ellos

✓ **Éxito:** comparar (medible) memoria/layout de `plain_data[]` vs un array de objetos.

G2 Escape analysis lite Cumbre

Cumbre Versión declarativa/estática (el automático completo es del JIT, Hito F3)

✓ **Éxito:** aplanar en *load time* cuando se prueba que un objeto no escapa.

Estado actual

Fase A / Hitos A0–A5 logrados; A6 en progreso. El proyecto compila **sin warnings**, pasa **113 tests**, cubre **el set de opcodes completo** (199 de 199 alcanzables, `invokedynamic` incluido), produce **fixtures byte-idénticos** a `javap` y ya **ejecuta bytecode real con objetos, GC generacional, hilos (green y de SO bajo un GIL), monitores y un sistema de tipos completo**: aritmética, control de flujo, recursión, `switch` (`tableswitch` / `lookupswitch`), un **heap** con objetos/herencia/campos/arrays, **dispatch dinámico**, **excepciones**, **inicialización de clases**, **class loaders**, un **recolector generacional** (young por copia + Old mark-sweep-compact, con **referencias débiles** de `java.lang.ref`), **green threads** (scheduler cooperativo) con **monitores** (`synchronized` de bloques y métodos, `wait` / `notify`), y un **verificador de bytecode JVMs-estricto**.

- **ClassReader** (cursor big-endian) y **constant pool** completo (`ConstantPoolEntry`, 17 tags + `Tombstone`), con árbol de referencias en `pretty_class_visualizer.rs`.
- **Header**: versiones, `access_flags` (+ métodos `is_*`), `this_class` / `super_class` con `class_name()`. `from_path()` → `Result` (sin panics).
- **Code** con **desensamblado completo** (opcodes + `tableswitch` / `lookupswitch` / `wide`) y comentarios `// ...` resueltos.
- **StackMapTable** (**los 7 frame types**), cada frame en su clase bajo `parser/stack_map_table/`, con `verification_type_info`.
- `javap` **brief** y **-v byte-idénticos** (incl. cabecera Classfile / Last modified / SHA-256). Flags de visibilidad (`-public` / `-protected` / `-package` / `-p`).
- Renderers factorizados en `parser/printers/`: `verbose` (orquestación) + `file_header` + `pool_comments` + `member_dump` + `brief` + `dump_common` + `visibility`.

Pendiente de A0 (no esencial, no bloquea avanzar): atributos `Signature` (reescribe la declaración → parser de firmas genéricas), `BootstrapMethods`, `InnerClasses` / `EnclosingMethod`, `Exceptions`, `ConstantValue`, anotaciones, `Record`, `PermittedSubclasses`, `NestHost` / `NestMembers`, `LocalVariableTable` / `Type`; y flags de contenido (`-c` / `-l` / `-s`).

A1–A2 — el intérprete. Loop de despacho sobre una **pila de frames**; cada frame referencia su bytecode por índice (`MethodId`) en el *Method Area* (`metaspace`, con resolución de llamadas por el código del constant pool). Opcodes: `iconst` / `iload` / `istore`, `iadd` / `isub` / `imul`, `goto` / `if_icmpgt`, `invokestatic` / `ireturn`. Corre **factorial** y **fibonacci recursivos**. Visualizador `jvm-step` paso a paso con vista de dos paneles de la call stack.

A3 — objetos y heap. El **heap** es una arena de bytes (`Vec<u8>`) con `malloc` bump (sin liberar; el GC llega en A5). Los objetos se disponen como `[class_id | mark | campos]` con *layout* que respeta la herencia; cada clase recibe un **Class ID** (UUID) y un *mirror* en el heap para sus estáticos (estilo HotSpot). Opcodes: `new`, `dup`, `getfield` / `putfield`, `aload` / `astore`, los cuatro `invoke*` y la familia de arrays (`newarray` / `anewarray`, `arraylength`, `iaload` / `iastore` / `aaload` / `aastore` + byte/char/short). El **dispatch dinámico** usa *vtables* por clase (slot del tipo estático, método del tipo real) e *itable* para interfaces — verificado: `Animal a = new Dog(); a.sound()` → `Dog.sound`.

A4 — robustez del runtime. Excepciones: `athrow` + tabla de excepciones + *stack unwinding* por la pila de frames, con `finally` (catch-all + re-throw) y las **implícitas** que la VM sintetiza (NPE, índice fuera de rango, cast inválido, tamaño negativo). **Verificador** de bytecode (type-checking en una pasada con el `StackMapTable`) que corre antes de ejecutar. **Inicialización** de clases `<clinit>` perezosa y super-primero. **Class loaders** bootstrap/app con delegación parent-first (las `java.lang.*` propias viven en `boot/`). Resolución que falla con **errores de linkage** (`NoClassDefFoundError` / `NoSuchMethodError`) en vez de paniquear. Pendiente fino: chequeos de acceso, identidad (`(nombre, loader)`), stack traces.

A5 — nativos + GC. Puente nativo (`System.out.println` imprime de verdad) e *intrinsic*s (`getClass`, `hashCode`, `Math`, `arraycopy`, `String` / `Class`). **Recolector de basura** que arrancó en mark & sweep y creció a un colector con **compactación** (mover objetos + reescribir punteros), **free list** con *coalescing*, **política de fragmentación** configurable, **disparadores** (out-of-space / occupancy / allocation-rate / explícito) sobre un *safe point*, y **marcado transitivo** correcto (sigue campos, arrays y estáticos).

A6 (en progreso) — cumbre del runtime. Verificador de bytecode JVMs-estricto: además de la cobertura completa de opcodes (incluido `switch`), verifica **con o sin** `StackMapTable` — inferencia por punto fijo como fallback, con *cross-check* de la tabla contra la inferencia — sobre un **gate estructural** (§4.9), con **tipos de locales** estrictos + `max_stack`, reglas de **objetos sin inicializar**, **handlers de excepción** tipados y **acceso/linkage** (regla `protected`, `count` de `invokeinterface`). **Sistema de tipos completo:** los cuatro tipos numéricos (`int` / `long` / `double` / `float`) **ejecutados y verificados** — cómputo, conversiones, comparaciones, división con `ArithmeticException`, **categoría-2** (params, campos, estáticos, arrays, frames del `StackMapTable`) y el **lattice de referencias** (covarianza de arrays + *join*/LUB). **GC generacional:** arena dividida en `Eden` | `S0` | `S1` | `Old`; los objetos nacen en Eden, un **minor** por **copia** evacúa los pocos sobrevivientes a un *survivor* (o los **promueve** a Old por edad) reescribiendo referencias con *forwarding*, un **write barrier** mantiene un **remembered set** de punteros `old→young` (las raíces que el minor escanea, sin recorrer todo Old), y el **major** hace mark-sweep-compact sobre Old. **Referencias débiles** (`java.lang.ref`): `WeakReference` + `ReferenceQueue` — el major trata el *referent* como débil y, al morir, lo limpia (`get()` → `null`) y **encola** la referencia; cimiento de `PhantomReference` / `Cleaner`.

Cobertura del set de opcodes — 199/199 alcanzables. El intérprete despacha todos los opcodes del JVMs que un class file moderno puede contener. Los últimos en cerrarse fueron `nop` (0x00), `goto_w` (0xc8) — el `goto` de offset de 4 bytes, para targets a más de ±32 KB, implementado ensanchando `jump_to` hacia un `jump_to_wide` común en vez de duplicar la aritmética — y el prefijo **wide** (0xc4), que reejecuta el opcode siguiente con un índice de local de **16 bits**: la única manera de direccionar los slots pasados el 255 en un método con más de 256 locales (`wide iinc` mide 6 bytes, el resto 4). Ese último salió barato por una propiedad que el diseño ya tenía: los handlers de `variable_operations` reciben `slot: use` y **nunca se enteran del ancho**, así que ensanchar es puro *decoding* y el módulo no cambió de comportamiento. Verificado por *differential testing*: `WideLocals.java` declara 300 locales para forzar a `javac` a emitir el prefijo (`istore_w 299`, `iinc_w 299, 35`, `iload_w 299`), y el mismo `.class` da 42 tanto en el `java` de JDK 25 como en nuestra VM. **De los 4 restantes, 3 son una decisión y no deuda:** `jsr` / `ret` / `jsr_w` están prohibidos por **JVMS §4.9.1** en class files de versión 50.0+ (Java 6 en adelante) — desde que existe el `StackMapTable` y el verificador por *type-checking*, las subrutinas quedaron fuera del modelo, y `javac` compila `finally` duplicando el bloque. La postura del proyecto es **leer sí, ejecutar no**: el desensamblador soporta `jsr` / `ret` / `jsr_w` / `ret_w` porque un `javap` debe renderizar *cualquier* class file legal (lo exige A0, medido sobre `java.base`), mientras el gate

estructural del verificador los rechaza y el intérprete no los despacha. Curiosamente *ejecutarlos* sería lo más fácil que queda —unas 10 líneas—; el costo está en una variante `ReturnAddress` que toca todos los `match` sobre `Value` y, sobre todo, en verificar subrutinas polimórficas, la parte más difícil del type-checker del JVM. Razonamiento completo en la nota de diseño `subrutinas-jsr-ret.md`. Contarlos como faltantes mezclaría una decisión con una tarea: el número honesto es **199 de 199 alcanzables — completo**.

invokedynamic (0xba) — el subsistema. El opcode solo no hace nada: no nombra ningún método, sino un *bootstrap* que **produce** el destino la primera vez. Esa indirección es lo que le permitió a Java agregar lambdas, concatenación de strings y records sin inventar un opcode por cada una — el lenguaje pone la política en el bootstrap y la VM queda fija. Corren **5 de las 6 fábricas** que emite `javac`:

`StringConcatFactory`, `SwitchBootstraps.typeSwitch` (patrones de tipo y de enum), `ObjectMethods` (los tres métodos de un `record`, desde una sola entrada de `BootstrapMethods` y distinguidos por el *nombre* del call site), `LambdaMetafactory.metafactory` y `ConstantBootstraps.invoke`. La sexta, `altMetafactory`, necesita serialización (`java.io`) y queda fuera de alcance. Dos decisiones de diseño: los *bootstrap methods* se modelan como **intrínsecos** —el mismo criterio de `intrinsecos.md`—, con

`java.lang.constant` escrito en **Java**; y las lambdas se resuelven con un **objeto sintético** en lugar de generar una clase en runtime. Esto último es el único lugar del proyecto donde nuestro diseño es *más simple* que el de HotSpot y no una concesión: el JDK genera clases porque la interfaz de la JVM le está fijada desde afuera; nosotros somos dueños del despacho. Ruta, correcciones y mediciones:

`invokedynamic-ruta.md`.

La VM puede invocar Java (`call_java`). Empuja un frame propio y lo corre con un bucle de `run_one` anidado, devolviendo el resultado. Resultó ser el **caso general de lo que la inicialización de clases venía haciendo**: `<clinit>` era la variante sin argumentos ni resultado, así que el mecanismo ya existía entero y sólo estaba especializado. Importa porque **los intrínsecos dejan de ser terminales**: un nativo que sólo puede calcular y devolver está obligado a reimplementar lo que necesite de la biblioteca — así fue como `String.valueOf` terminó a medias en Rust. Con esto, `String.valueOf(Object)` llama al `toString()` del objeto, un `record` pregunta el `equals` / `hashCode` de sus componentes en vez de comparar referencias, y una constante dinámica ejecuta su bootstrap. Lo que falta para cerrar 0xba del todo es el **modelo de objetos de `java.lang.invoke`** (`MethodHandle` / `MethodType` / `Lookup`): las clases irían en Java, pero `MethodHandle.invoke` es nativo por necesidad —invocar un método arbitrario desde un handle es justo lo que Java no puede hacerse a sí mismo, y el JDK real también lo trata así—. Parte de eso espera al escritor de `.class` (hito B3), porque `javac` nunca emite un `ldc` de esas constantes ni permite invocar un bootstrap propio.

multianewarray (0xc5). El último en cerrarse, y el que más enseña sobre el modelo de datos: **Java no tiene arrays rectangulares**. `new int[2][3]` no es un bloque plano sino dos `[I` colgando de un `[[I`, cada nivel un objeto real — por eso las filas se reemplazan por separado y `a[0].length` no tiene por qué igualar `a[1].length`. La instrucción lleva el descriptor del array *mismo* (`[[I`, no el elemento como `anewarray`) más un contador de dimensiones, porque sólo se materializan **esos** niveles: `new int[3][]` deja los internos en `null`. Los conteos se validan *todos* antes de alocar nada, para que un largo negativo en una dimensión posterior no deje un array a medio construir; los niveles superiores guardan referencias y sólo el más interno usa el ancho real del elemento (una fila `byte[]` ocupa un byte por elemento, no cuatro); y las referencias hijo se escriben por `store_reference`, nunca por un `write_u32` crudo, porque son exactamente los punteros `old→young` que el *remembered set* debe registrar.

Ejecutarlo era la mitad del trabajo: hubo que modelarlo en el `transfer` del verificador, ya que un opcode sin modelar produce un `VerifyError` marcado `unsupported` — que por diseño hace que el llamador avise y siga, o sea que la clase habría corrido con el verificador claudicando en silencio.

Green threads (Fase 0). `java.lang.Thread` + `Thread.start()` nativo, un scheduler **cooperativo** round-robin que conmuta en el safepoint (entre opcodes), un call stack por-thread, y el GC tomando las raíces de **todos** los threads. Verificado: dos workers intercalándose con `main` (que los espera por spin-wait) dan `100`, y el step-visualizer muestra los cambios de contexto, el spawn y la terminación de cada thread. Es el modelo de los *virtual threads* de Java 21 (scheduling en espacio de usuario), con un solo carrier.

Hilos de SO + GIL (E1+E2). El substrato es un **parámetro de aplicación** (`JVM_THREADS`, como los `JVM_GC_*`): en modo `os` cada `Thread.start()` lanza un `std::thread` real y la VM entera vive tras un **GIL** (`Arc<Mutex<JVM>>`) que se toma y se suelta **por opcode**, en el mismo safepoint que ya usaba el GC. El bloqueo es `park` / `unpark` de verdad (centralizado en `make_runnable`) para monitores, `wait` / `notify` y `join`, y `Thread.sleep` pasa a tiempo real. El GC corre seguro porque el GIL es el stop-the-world: un solo hilo toca la VM a la vez, así que heap, monitores y metaspace quedan correctos sin sincronización extra. Es el camino canónico — el de CPython y las JVM tempranas —: **correcto pero todavía serializado**. Validado con 4 tests de modo OS que dan el mismo resultado que en green, usando el intérprete cooperativo como oráculo de corrección. **El costo real de este salto no fue el JIT sino el refactor de ownership**, y lo abarató el seam: toda escritura de referencia ya pasaba por `HeapService.store_reference`.

Monitores. Cada objeto tiene su monitor intrínseco (lock reentrante con dueño + *blocked-set* + *wait-set*). `synchronized` funciona en sus dos formas: **bloques** (opcodes `monitorenter` / `monitorexit`) y **métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma el monitor del receptor o del `Class` al empujar el frame y lo suelta al popearlo, por `return` o por *unwind* de excepción, vía un único punto de release imposible de saltar). La señalización `wait` / `notify` / `notifyAll` libera el monitor (guardando el conteo de reentrada), parquea al hilo en el *wait-set* y lo re-adquiere al despertar. Verificado con exclusión mutua real (dos hilos × 100 incrementos → 200, contra el control sin lock que pierde updates) y el *handshake* productor/consumidor (`wait` → `notify` → 42). Después se cerró el resto:

`IllegalMonitorStateException` (gate `owns_monitor` sobre `monitorexit` / `wait` / `notify`, JVMs §6.5), `Object.wait(long)` con **timeout** (deadline sobre `sleep_until`; reloj de opcodes en green, tiempo real en OS — demo `WaitTimeout` → 7) y el **monitor GC-safe**: los monitores se keyean por offset del heap, así que al mover objetos (minor y compact) las claves se remapean por el *forward* del GC, y los monitores de objetos colectados se descartan — ya se puede compactar con monitores tomados (demo `GcMonitor` → 5, y el test se cuelga si se quita el remapeo). **Pendiente del monitor:** la interrupción del `wait` (llega con H1), biased/thin locks y detección de deadlock.

Arquitectura en capas (estilo *service*). El heap se encapsuló tras un `HeapService` que es su **único dueño**: toda escritura de referencia pasa por un portón (`store_reference`) que aplica el **write barrier** de forma no-bypasseable — el seam donde irá la sincronización cuando los threads pasen a ser de SO. El runtime es la `JVM` (composition root) sobre `HeapService` + `MetaspaceService` + el sistema de threads.

✓ **Siguiente: H1 — la API de `Thread`** (ver la sección «Concurrencia»): extender `ThreadStatus` a los seis estados de `Thread.State`, sumar `currentThread` / `yield` / `getState` / `isAlive` y `Thread(Runnable)`, y cerrar `interrupt` / `InterruptedException` — que de paso completa la última pieza del monitor. Después,

E3: sacar el GIL (locks finos + TLABs + handshake stop-the-world → Java en paralelo) → modelo de memoria (`volatile` /fences) → CAS → `java.util.concurrent` . El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA).